

# 1. INTRODUCTION

## 1.1 Background

In contemporary healthcare systems, the widespread adoption of Electronic Health Record (EHR) and Electronic Medical Record (EMR) systems has resulted in the generation of massive volumes of clinical documents. Healthcare providers generate extensive unstructured text narratives daily, including admission notes, consultation records, operative reports, discharge summaries, and diagnostic evaluations. Categorizing clinical text into specific medical specialties (e.g., Cardiology, Neurology, Orthopedics, Gastroenterology, Surgery) is a fundamental prerequisite for organizing health databases, routing clinical files, and supporting decision systems.

Traditionally, this categorization relies on manual review by medical coders and health information management staff. As patient volumes increase, manual document indexing becomes increasingly bottlenecked, leading to administrative delays and human classification errors. Natural Language Processing (NLP) and Machine Learning (ML) present viable computational paradigms for automating clinical text categorization, enabling rapid and consistent medical specialty identification.

## 1.2 Problem Statement

Manual categorization of medical transcription reports into clinical specialties is time-consuming, labor-intensive, and highly prone to inter-annotator variability. Unstructured clinical notes exhibit complex linguistic characteristics, including domain-specific jargon, non-standard medical abbreviations, variable document lengths, and semantic ambiguity. Furthermore, real-world clinical text datasets suffer from severe class imbalance, where dominant medical specialties contain thousands of records, whereas specialized sub-disciplines contain very few. Existing systems lack computationally efficient mechanisms to handle high-dimensional clinical text spaces while remaining lightweight enough for real-time web deployment.

## 1.3 Need for the System

Automating medical specialty classification yields key operational and clinical benefits:

- *Streamlined Workflow*: Eliminates manual tagging backlogs, allowing clinical reports to

be indexed and routed immediately upon dictation.

- *Enhanced Information Retrieval*: Enables clinicians to query specialty-specific diagnostic records rapidly during emergency care.
- *Reduced Administrative Overhead*: Allows medical records staff to focus on quality assurance rather than manual document sorting.
- *Scalable Health Analytics*: Provides standardized metadata necessary for hospital resource allocation and epidemiological research.
- *Real-Time Accessibility*: Proposes an intuitive web interface for healthcare professionals to upload reports and receive specialty predictions.

## 1.4 Objectives

The primary objective of this project is to develop and deploy an automated Medical Specialty Classification system using TF-IDF feature representations and an Ensemble Machine Learning architecture. The specific objectives are:

1. To conduct Exploratory Data Analysis (EDA) on the Kaggle Medical Transcriptions (MTSamples) dataset.
2. To build an NLP preprocessing pipeline incorporating lowercasing, punctuation removal, tokenization, stop-word removal, and lemmatization.
3. To extract numerical text features using Term Frequency–Inverse Document Frequency (TF-IDF) vectorization.
4. To train and evaluate candidate machine learning models (Logistic Regression, Support Vector Machine, Random Forest, Multinomial Naive Bayes).
5. To construct a Voting Ensemble Classifier combining predictions from candidate base models.
6. To deploy a real-time, interactive web application using Flask for medical document classification.

## 1.5 Scope

The scope of this project encompasses:

- *Dataset Scope*: Evaluates 4,999 clinical transcription records across 40 medical specialties from the Kaggle MTSamples dataset.
- *Feature Scope*: Focuses on TF-IDF vectorization derived from the primary narrative column (transcription).
- *Algorithmic Scope*: Evaluates classical ML models (LR, SVM, RF, MNB) and a Voting Ensemble Classifier optimized for CPU deployment.
- *Deployment Scope*: Proposes a lightweight Flask web interface for single-document and batch specialty prediction.

## 1.6 Project Overview

The project follows a structured engineering workflow:

1. *Data Ingestion & EDA*: Ingesting raw MTSamples CSV data and analyzing feature distributions.
2. *Text Preprocessing*: Removing noise, tokenizing, filtering stop-words, and lemmatizing text.
3. *Feature Engineering*: Transforming clean text into sparse numerical vectors via TF-IDF.
4. *Model Development*: Training baseline classifiers and hyperparameter tuning.
5. *Ensemble Integration*: Combining the outputs of the candidate models using soft voting to generate the final specialty prediction.
6. *Web Deployment*: Serializing model artifacts with Joblib and serving via a proposed Flask web application.

## 1.7 Expected Outcome

The expected deliverables include:

- A clean, preprocessed clinical text dataset optimized for NLP tasks.
- A proposed Voting Ensemble Classifier targeting robust performance across 40 medical specialties.
- A planned lightweight Flask web application to provide real-time specialty classification.
- Complete technical documentation adhering to MCA departmental standards.

## 2. SUPPORTING LITERATURE

### 2.1 Literature Review

A systematic review of contemporary literature was conducted to evaluate existing methodological approaches, feature representations, classification algorithms, and dataset challenges in medical text categorization. The review relies strictly on approved reference literature in clinical natural language processing.

**Paper 1 – Omar AM, Elhoseny M, Hassanien AM. Clinical text classification with word representation features and machine learning algorithms. International Journal of Online and Biomedical Engineering (iJOE). 2023.**

The paper titled “Clinical Text Classification with Word Representation Features and Machine Learning Algorithms” evaluates the impact of different feature extraction techniques and machine learning algorithms on Electronic Medical Record (EMR) narratives. The authors introduced a four-phase classification framework consisting of text preprocessing, word vectorization, feature reduction, and supervised classification.

To model clinical text narratives, the study compared classical bag-of-words and TF-IDF representations against dense neural embeddings (Word2Vec) across multiple classifiers, establishing baseline benchmarks for clinical text processing. The evaluated classifiers include Logistic Regression, Support Vector Machine (SVM), Naive Bayes, and k-Nearest Neighbors (k-NN).

The empirical results demonstrated that combining Word2Vec embeddings with a k-NN classifier achieved the highest accuracy of 92%. Dense semantic embeddings consistently outperformed sparse unigram Bag-of-Words representations across clinical text categories. However, distance-based k-NN introduces significant computational latency during inference, rendering it inefficient for massive production databases. Furthermore, the study relied on single classifiers without exploring ensemble learning strategies or real-time web deployment frameworks. Crucially, Principal Component Analysis (PCA) was evaluated exclusively in Paper 1 for feature reduction and is not included in our proposed system architecture.

|                                |                                                                                                                                  |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>TITLE</b>                   | Clinical Text Classification with Word Representation Features and Machine Learning Algorithms [1]                               |
| <b>AREA OF WORK</b>            | Clinical Natural Language Processing / EMR Text Vectorization                                                                    |
| <b>DATASET</b>                 | Electronic Medical Record (EMR) Clinical Transcriptions Dataset                                                                  |
| <b>METHODOLOGY / STRATEGY</b>  | 4-Phase Framework: (1) Preprocessing, (2) Vectorization (BOW, TF-IDF, Word2Vec), (3) Feature Reduction (PCA), (4) Classification |
| <b>ALGORITHM</b>               | Logistic Regression, Support Vector Machine (SVM), Naive Bayes, k-NN, PCA                                                        |
| <b>RESULT / PRECISION</b>      | Word2Vec + k-NN achieved the highest classification accuracy of 92%                                                              |
| <b>ADVANTAGES</b>              | Proves dense semantic embeddings capture medical context better than unigram BOW                                                 |
| <b>LIMITATIONS</b>             | Distance-based k-NN scales poorly; lacks ensemble learning and web UI; PCA used only in literature                               |
| <b>RELEVANCE TO MY PROJECT</b> | Establishes the standard 4-phase text preprocessing and feature extraction pipeline                                              |
| <b>FUTURE PROPOSAL</b>         | Develop lightweight ensemble classifiers suitable for real-time web deployment                                                   |

**Table 1:** Summary of Paper 1

**Paper 2 – Zhang H, Zhu D, Tan H, Shafiq M, Gu Z. Medical specialty classification based on semiadversarial data augmentation. Hindawi Computational Intelligence and Neuroscience. 2023.**

The research article titled “Medical Specialty Classification Based on Semiadversarial Data Augmentation” addressed data scarcity and category imbalance in medical specialty classification. The authors developed Semiadversarial Data Augmentation (SemiADA), generating synthetic samples along adversarial attack trajectories to populate sparse decision regions.

Additionally, a Probabilistic Information (PI) layer was introduced post-Softmax to recalculate confidence based on domain noun importance. The paper evaluated a filtered subset of the Kaggle Medical Specialty Classification dataset restricted to 18 medical specialty classes. In contrast, our project utilizes the complete Kaggle Medical Transcriptions (MTSamples) dataset spanning 40 medical specialties.

The proposed SemiADA framework achieved an average performance improvement of 14.9% in accuracy and F1 score over baseline benchmarks. The study proved that domain-specific nouns carry the highest discriminative signal for medical specialty assignment. However, generating synthetic text along adversarial attack paths incurs extreme computational overhead. Fine-tuning heavy transformer models like BERT requires dedicated GPU infrastructure, making real-time inference difficult in resource-constrained web environments.

|                                |                                                                                                                           |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>TITLE</b>                   | Medical Specialty Classification Based on Semiadversarial Data Augmentation [2]                                           |
| <b>AREA OF WORK</b>            | Clinical Text Data Augmentation / Class Imbalance Handling                                                                |
| <b>DATASET</b>                 | Kaggle Medical Specialty Classification Dataset (Subset restricted to 18 classes)                                         |
| <b>METHODOLOGY / STRATEGY</b>  | Semiadversarial sample generation along attack paths + Probabilistic Information (PI) noun-weighted Softmax recalculation |
| <b>ALGORITHM</b>               | SemiADA, BERT (Transformer), Softmax with PI Noun Layer                                                                   |
| <b>RESULT / PRECISION</b>      | Achieved an average 14.9% boost in Accuracy and F1 score over baseline benchmarks                                         |
| <b>ADVANTAGES</b>              | Effectively handles extreme class imbalance using geometry-aware synthetic samples                                        |
| <b>LIMITATIONS</b>             | Extreme computational latency; fine-tuning BERT demands high-end GPU hardware                                             |
| <b>RELEVANCE TO MY PROJECT</b> | Identifies class imbalance in MTSamples data; motivates TF-IDF + Ensemble ML alternative                                  |
| <b>FUTURE PROPOSAL</b>         | Combine TF-IDF term weighting with ensemble learning to classify 40 specialties on standard CPUs                          |

**Table 2:** Summary of Paper 2

**Paper 3 – Blanchard AE, Gao S, Yoon HJ, Tourassi G. A keyword-enhanced approach to handle class imbalance in clinical text classification. IEEE Journal of Biomedical and Health Informatics. 2022.**

Blanchard et al., writing in the *IEEE Journal of Biomedical and Health Informatics*, focused on preventing deep neural models from overfitting on rare clinical categories in long pathology narratives. The authors introduced a data-centric strategy by injecting short class-associated

keyword sequences into mini-batches during neural network training, formulating a combined loss function

$$L = L_{\text{docs}} + \alpha L_{\text{key}}$$

The study utilized the National Cancer Institute (NCI) Surveillance, Epidemiology, and End Results (SEER) cancer pathology registry dataset. The approach evaluated Convolutional Neural Networks (CNNs), keyword-constrained loss functions, and standard data resampling baselines (oversampling and undersampling).

Injecting keyword constraints significantly improved macro F1 score and recall on rare classes. The study demonstrated that overall micro-accuracy is a deceptive metric in clinical NLP because high overall scores can mask near-zero recall on rare clinical conditions. However, the approach requires manual curation of comprehensive keyword dictionaries by clinical domain experts for every target category, limiting automated scalability to new medical domains.

|                                |                                                                                                         |
|--------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>TITLE</b>                   | A Keyword-Enhanced Approach to Handle Class Imbalance in Clinical Text Classification [3]               |
| <b>AREA OF WORK</b>            | Imbalanced Clinical Narrative Classification / Cancer Registry NLP                                      |
| <b>DATASET</b>                 | NCI SEER Surveillance Cancer Pathology Registry Reports Dataset                                         |
| <b>METHODOLOGY / STRATEGY</b>  | Ingesting keyword sequences into mini-batches during training to constrain the neural loss function     |
| <b>ALGORITHM</b>               | Convolutional Neural Networks (CNNs), Keyword-Constrained Loss Optimization                             |
| <b>RESULT / PRECISION</b>      | Significantly improved macro F1 score and recall on rare classes without degrading majority performance |
| <b>ADVANTAGES</b>              | Prevents deep models from memorizing noisy tokens in long text documents with rare labels               |
| <b>LIMITATIONS</b>             | Requires manual domain expert curation of keyword dictionaries for every target category                |
| <b>RELEVANCE TO MY PROJECT</b> | Demonstrates that keyword/n-gram features preserve rare class signals, supporting TF-IDF                |
| <b>FUTURE PROPOSAL</b>         | Utilize automated TF-IDF n-gram feature extraction without requiring manual dictionary creation         |

**Table 3:** Summary of Paper 3

## 2.2 Literature Review Summary

Table 4 presents a concise summary of the reviewed literature alongside our proposed system.

| Paper                | Area                            | Dataset                       | Algorithm                       | Advantages                       | Relation to Proposed Work                  |
|----------------------|---------------------------------|-------------------------------|---------------------------------|----------------------------------|--------------------------------------------|
| Omar et al. [1]      | Clinical Vectorization          | EMR Text                      | LR, SVM, NB, k-NN               | High accuracy (92%)              | Establishes 4-phase preprocessing pipeline |
| Zhang et al. [2]     | Data Augmentation               | MTSamples (18 classes)        | SemiADA, BERT                   | +14.9% accuracy gain             | Highlights MTSamples class imbalance       |
| Blanchard et al. [3] | Class Imbalance                 | NCI SEER Pathology            | CNN + Keyword Loss              | High macro F1 on rare classes    | Justifies TF-IDF term weighting            |
| <b>Proposed Work</b> | <b>Specialty Categorization</b> | <b>MTSamples (40 classes)</b> | <b>TF-IDF + Voting Ensemble</b> | <b>Fast, lightweight, web UI</b> | <b>Complete MCA Project System</b>         |

**Table 4:** Overall Summary of all paper

### Evolution of Research

Clinical text processing research has transitioned from basic Bag-of-Words word counting toward neural word embeddings (Word2Vec), transformer architectures (BERT), and complex adversarial data augmentation (SemiADA). While deep neural networks achieve strong performance, their high computational cost and memory footprints prevent real-time deployment in standard healthcare web environments.

### Identified Research Gaps

The literature review reveals three primary research gaps:

1. **High Computational Overhead:** Existing deep learning systems require dedicated GPUs and introduce substantial inference latency.
2. **Lack of Real-Time Web Deployment:** Prior studies focus exclusively on offline evalua-

tion metrics without building accessible user interfaces.

3. **Single Model Reliance:** Most prior works rely on single classifiers rather than leveraging multi-model ensemble diversity.

## **Motivation for Proposed System**

Our project addresses these research gaps by combining TF-IDF vectorization with a Voting Ensemble Classifier (aggregating Logistic Regression, Linear SVM, Random Forest, and Multinomial Naive Bayes) intended for deployment via a lightweight Flask web application. This architecture is designed to target accurate classification across all 40 medical specialties in the full MTSamples dataset while aiming for efficient CPU inference for real-world clinical use.

## **2.3 Findings and Proposals**

### **2.3.1 Findings**

The review of the existing literature shows that machine learning techniques can effectively classify medical and clinical text into different medical specialties. Traditional machine learning algorithms such as Logistic Regression, Support Vector Machine, Random Forest, and Naive Bayes have demonstrated good performance in text classification tasks when appropriate feature extraction techniques are used.

The literature also indicates that TF-IDF feature extraction is effective for converting unstructured clinical text into numerical feature vectors. It helps identify important medical terms and provides suitable high-dimensional input for machine learning algorithms.

Another important finding is that the performance of individual machine learning models may vary depending on the characteristics of the dataset. Some algorithms perform better for certain classes, while others provide better overall generalization. Therefore, relying on a single classifier may not always produce the best prediction performance.

The reviewed studies further show that ensemble learning techniques, which combine predictions from multiple machine learning models, can improve the robustness and reliability of classification systems. However, challenges such as class imbalance, noisy clinical text, missing values, and variations in medical terminology need to be considered during data preprocessing. Based on these findings, it is observed that combining effective text preprocessing, TF-IDF fea-

ture extraction, multiple machine learning algorithms, and an ensemble classification approach can provide a suitable solution for medical specialty classification.

### **2.3.2 Proposal**

Based on the findings from the literature review, the proposed system aims to develop a Medical Specialty Classification system using TF-IDF and Ensemble Machine Learning.

The system will use clinical transcription data containing medical reports and their corresponding medical specialties. Initially, the dataset will be analysed and preprocessed by handling missing values, removing unnecessary information, converting text into a consistent format, and cleaning the clinical transcription data.

After preprocessing, the clinical text will be transformed into numerical feature vectors using the TF-IDF Vectorizer. Multiple machine learning algorithms, including Logistic Regression, Linear Support Vector Machine, Random Forest, and Multinomial Naive Bayes, will then be trained using the extracted features.

The predictions generated by the individual machine learning models will be combined using an ensemble learning approach, such as Voting Classification. The proposed approach is expected to improve the reliability and classification performance compared with depending on a single machine learning model.

Finally, the trained system will accept a clinical transcription as input and predict the most appropriate medical specialty. The system will be evaluated using suitable performance measures such as accuracy, precision, recall, and F1-score.

Thus, the proposed system combines text preprocessing, TF-IDF feature extraction, multiple machine learning algorithms, and ensemble learning to provide an efficient approach for the automatic classification of medical clinical transcriptions.

### 3. SYSTEM ANALYSIS

System analysis establishes the functional requirements, dataset architecture, preprocessing steps, exploratory data analysis, and algorithmic framework for the medical specialty classification system.

The Kaggle Medical Transcriptions (MTSamples) dataset, sourced from Kaggle [4], provides clinical narrative dictations across multiple healthcare sub-disciplines, enabling comprehensive evaluation of natural language processing and machine learning models for medical document categorization.

**Dataset Link:** <https://www.kaggle.com/datasets/tboyle10/medicaltranscriptions>

**Size:** The dataset comprises 4,999 clinical records spanning 40 distinct medical specialties.

**Data Structure:** It contains six attributes including row index (Unnamed: 0), short report abstract (description), target medical department (medical\_specialty), sample dictation title (sample\_name), main clinical narrative (transcription), and extracted key terms (keywords).

**Sparsity:** The primary text column (transcription) is highly complete, containing 33 missing records (0.66%), which are dropped during preprocessing. The target label (medical\_specialty) contains zero missing values.

#### 3.1 EXPLORATORY DATA ANALYSIS

##### 3.1.1 ABOUT THE DATASET

###### Dataset Overview

The Kaggle Medical Transcriptions dataset provides authentic clinical narratives dictating patient histories, diagnostic evaluations, physical examinations, and surgical procedures:

- Total Records: 4,999 clinical reports.
- Total Features: 5 independent variables + 1 target variable (medical\_specialty).
- Data Types: Textual narratives (transcription, description, keywords), categorical labels (medical\_specialty), and integer index.
- Missing Values: Present in description (6 records), transcription (33 records), and keywords (1,068 records).

Table 5 summarizes the core dataset attributes.

| Property                 | Description                                                                                                                                            |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dataset Name</b>      | Kaggle Medical Transcriptions (MTSamples) Dataset                                                                                                      |
| <b>Source</b>            | Kaggle ( <a href="https://www.kaggle.com/datasets/tboyle10/medicaltranscriptions">https://www.kaggle.com/datasets/tboyle10/medicaltranscriptions</a> ) |
| <b>Total Records</b>     | 4,999 clinical reports                                                                                                                                 |
| <b>Number of Columns</b> | 6 fields (Unnamed: 0, description, medical_specialty, sample_name, transcription, keywords)                                                            |
| <b>Input Variable</b>    | transcription (Primary clinical narrative text)                                                                                                        |
| <b>Target Variable</b>   | medical_specialty (40 unique clinical specialties)                                                                                                     |
| <b>Advantages</b>        | Authentic clinical dictations; complete target label annotations (0 missing labels); broad multi-class domain coverage                                 |
| <b>Limitations</b>       | Severe class imbalance across categories; 33 missing transcriptions; 1,068 missing keyword entries; variable text length                               |

**Table 5:** Characteristics of the Kaggle Medical Transcriptions Dataset

The dataset attributes are categorized as follows:

1. **Primary Clinical Narrative** (transcription): The central predictor attribute containing unstructured, dictation-style medical reports that detail symptoms, findings, diagnoses, and treatments.
2. **Clinical Abstract** (description): Short one-to-two sentence summary of the clinical consultation or procedure.
3. **Extracted Keywords** (keywords): Comma-delimited clinical terms and anatomical references extracted from the narrative.
4. **Sample Identification** (sample\_name): Dictation title indicating the type of procedure or medical note.
5. **Target Variable** (medical\_specialty): The multi-class categorical label representing the specialized department (40 clinical specialties).

### 3.1.2 STATISTICAL ANALYSIS

Data exploration was conducted using Python data analysis tools (pandas, numpy). Inspection via shape, info(), dtypes, and describe() confirmed that the dataset contains 4,999 records and 6 columns without structural corruption. Table 6 describes each column, its data type, and missing value counts.

| Column Name       | Data Type   | Description                           | Missing Count |
|-------------------|-------------|---------------------------------------|---------------|
| Unnamed: 0        | Integer     | Record Identifier / Row Index         | 0             |
| description       | Text        | Short summary of clinical report      | 6             |
| medical_specialty | Categorical | Target specialty label (40 classes)   | 0             |
| sample_name       | Text        | Report title / dictation sample title | 0             |
| transcription     | Text        | Primary clinical narrative (Input)    | 33            |
| keywords          | Text        | Extracted key terms                   | 1,068         |

**Table 6:** Dataset Column Descriptions and Missing Value Audit

Descriptive statistical analysis revealed significant variation in class distribution across the 40 medical specialties. Table 7 lists the record counts for the top 10 majority specialties alongside the 10 rarest minority specialties.

| Top 10 Specialties            | Count | Bottom 10 Specialties      | Count |
|-------------------------------|-------|----------------------------|-------|
| Surgery                       | 1,103 | Cosmetic / Plastic Surgery | 27    |
| Consult - History and Phy.    | 516   | Pediatric / Neonatal       | 22    |
| Cardiovascular / Pulmonary    | 372   | Sleep Medicine             | 20    |
| Orthopedic                    | 355   | Bariatrics                 | 18    |
| Radiology                     | 273   | Speech - Language          | 9     |
| General Medicine              | 259   | Lab Medicine - Pathology   | 8     |
| Gastroenterology              | 230   | Allergy / Immunology       | 7     |
| Neurology                     | 223   | Hospice - Palliative Care  | 6     |
| SOAP / Chart / Progress Notes | 166   | Executive Evaluation       | 2     |
| Obstetrics / Gynecology       | 160   | Autopsy                    | 2     |

**Table 7:** Distribution of Top 10 Majority and Bottom 10 Minority Medical Specialties

Statistical analysis of narrative text lengths indicated a positively skewed distribution. While the median document length is approximately 350 to 500 words, certain comprehensive opera-

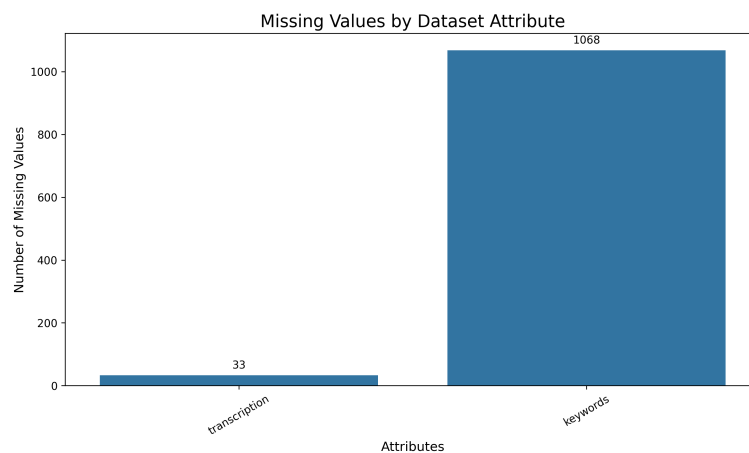
tive notes and consultative evaluations extend beyond 2,000 words. This variation in document length further supports the use of TF-IDF feature representation with vector normalization for consistent numerical feature scaling.

## 3.2 DATA VISUALIZATION

Data visualization is an important stage of Exploratory Data Analysis (EDA) that helps in understanding the characteristics, distributions, and relationships among clinical narratives and medical specialties. Visual representations provide valuable insights into the dataset, reveal hidden patterns, and assist in verifying the suitability of the text data for machine learning classification.

### Missing Value Analysis

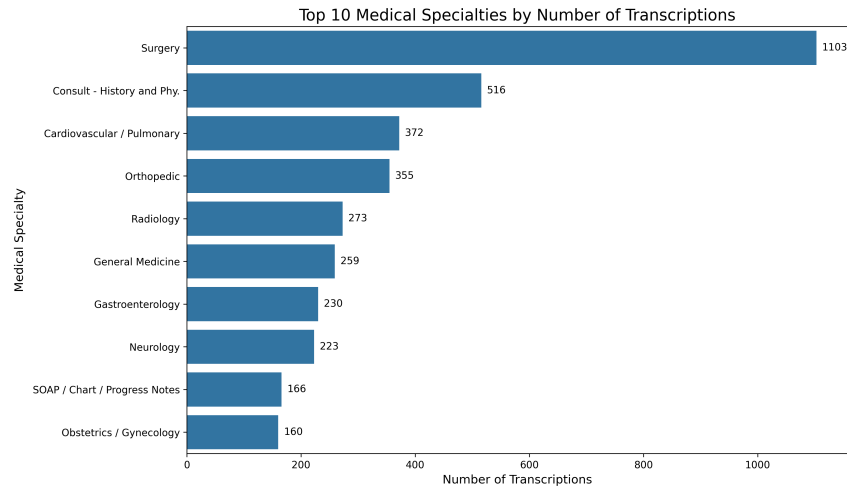
Figure 1 illustrates the distribution of missing values across dataset attributes. As shown, the primary target attribute (`medical_specialty`) contains zero missing values. The primary input text narrative (`transcription`) contains 33 missing records, while the auxiliary keywords attribute exhibits 1,068 missing records.



**Figure 1:** Missing Values by Dataset Attribute

### Top 10 Medical Specialties

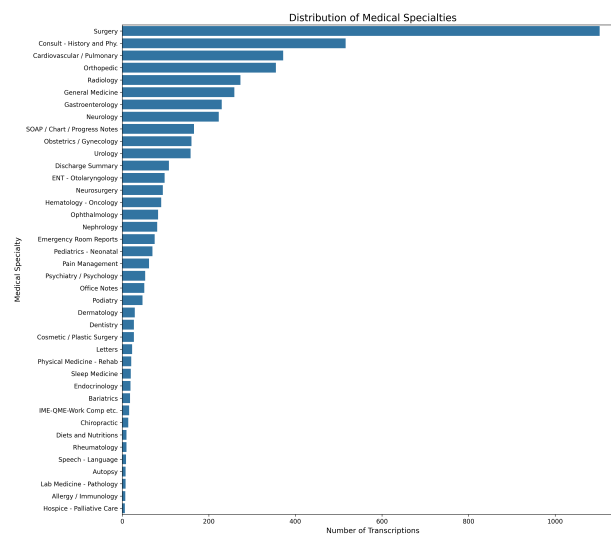
Figure 2 displays the record counts for the top 10 medical specialties in the dataset. Surgery represents the single largest category with 1,103 clinical reports, followed by Consult - History and Phy. (516 reports), Cardiovascular / Pulmonary (372 reports), and Orthopedic (355 reports).



**Figure 2:** Top 10 Medical Specialties by Number of Transcriptions

### Complete Specialty Distribution

Figure 3 illustrates the full class distribution across all 40 medical specialties in the MTSamples corpus. The chart demonstrates severe class imbalance, ranging from majority classes such as Surgery (1,103 records) down to highly specialized minority classes such as Hospice - Palliative Care, Allergy / Immunology, Lab Medicine - Pathology, and Autopsy with under 10 records each.

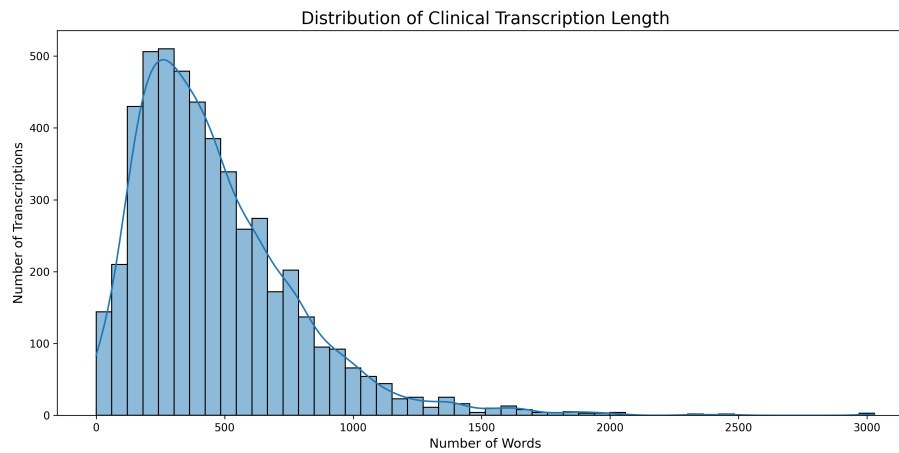


**Figure 3:** Distribution of Medical Specialties

### Transcription Length Distribution

Figure 4 displays the word count distribution for clinical narratives, fitted with a Kernel Density Estimate (KDE) curve. The majority of medical transcriptions range between 200 and 600

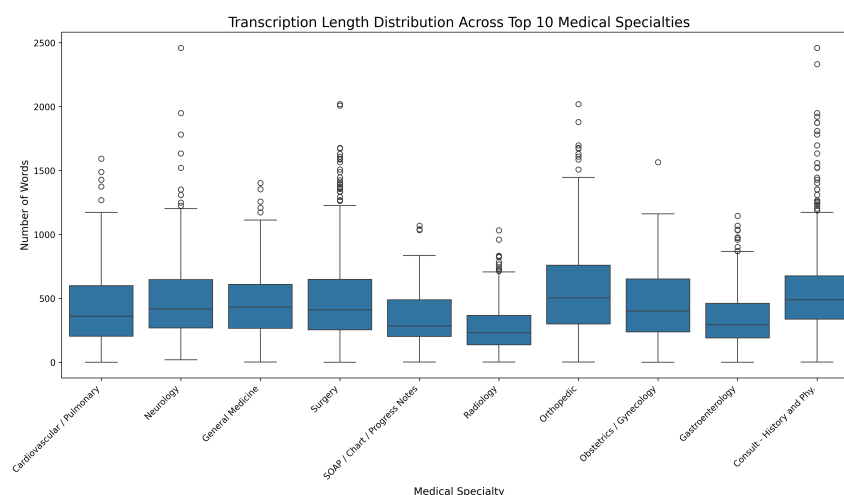
words, exhibiting a right-skewed distribution with extended narratives reaching up to 3,000 words.



**Figure 4:** Distribution of Clinical Transcription Length

### Transcription Length Across Top Specialties

Figure 5 presents box plots comparing narrative word count distributions across the top 10 medical specialties. Specialties such as Radiology and Gastroenterology feature shorter median narrative lengths ( 250 words), whereas Orthopedic, Surgery, and Consult - History and Phy. contain longer document narratives (median 500 words) with multiple high-length outliers exceeding 1,500 to 2,500 words.



**Figure 5:** Transcription Length Distribution Across Top 10 Medical Specialties

### Medical Word Cloud Visualization

Figure 6 displays a word cloud of the most frequent clinical terms extracted from the medical transcription corpus. High-frequency domain terms include "placed", "well", "used", "time",



### 3.3 DATA PREPROCESSING

Data preprocessing is an essential stage in developing a machine learning model, as it transforms raw, unstructured clinical narratives into a structured and meaningful format suitable for mathematical modeling. In this project, the original dataset consisted of free-text clinical dictations containing clinical terminology, punctuation, formatting artifacts, and varying narrative lengths. The preprocessing stage included data cleaning, text normalization, analysis of feature variables, and examination of the target class distribution.

#### 3.3.1 DATA CLEANING

Text preprocessing is essential in clinical NLP to remove noisy elements, normalize medical vocabulary, and transform raw narrative text into clean input suitable for mathematical vectorization. The cleaning pipeline executes eight sequential operations:

1. *Removing Missing Values*: Records containing empty or null transcription fields (33 records) are dropped from the dataset.
2. *Removing Duplicates*: Verification ensures no duplicate clinical reports exist in the corpus.
3. *Lowercasing*: All narrative text is converted to lowercase to standardize character representations.
4. *Removing Punctuation*: Commas, periods, semicolons, and special punctuation symbols are stripped.
5. *Removing Special Characters*: Numeric digits and non-alphanumeric symbols are removed.
6. *Tokenization*: Free-text narratives are segmented into individual word tokens.
7. *Stop-Word Removal*: Generic English stop-words are filtered out while preserving clinical vocabulary.
8. *Lemmatization*: Word tokens are mapped to their canonical base form using the WordNet Lemmatizer.

Table 8 provides concrete text transformation examples for each step.

| Preprocessing Step | Input Text Example                                   | Processed Output Example                             |
|--------------------|------------------------------------------------------|------------------------------------------------------|
| Raw Input Text     | "SUBJECTIVE: Patient suffers from acute chest pain!" | "SUBJECTIVE: Patient suffers from acute chest pain!" |
| 1. Missing Values  | Checked: Non-null text                               | Retained                                             |
| 2. Duplicates      | Checked: Unique entry                                | Retained                                             |
| 3. Lowercasing     | "SUBJECTIVE: Patient suffers..."                     | "subjective: patient suffers from acute chest pain!" |
| 4. Punctuation     | "subjective: patient suffers..."                     | "subjective patient suffers from acute chest pain"   |
| 5. Special Chars   | "chest pain! 100%"                                   | "chest pain"                                         |
| 6. Tokenization    | "patient suffers pain"                               | ['patient', 'suffers', 'pain']                       |
| 7. Stop-Words      | ['patient', 'suffers', 'from', 'pain']               | ['patient', 'suffers', 'pain']                       |
| 8. Lemmatization   | ['patient', 'suffers', 'pain']                       | ['patient', 'suffer', 'pain']                        |

**Table 8:** Step-by-Step Clinical Text Preprocessing Examples

### 3.3.2 ANALYSIS OF FEATURE VARIABLES

The transcription field is selected as the primary feature variable because it contains the complete clinical narrative. Other metadata fields, such as description and keywords, are not included in the primary classification feature set because the project focuses on full transcription-based specialty classification and the keywords field contains substantial missing values. Unstructured clinical narratives are converted into high-dimensional numerical feature vectors using Term Frequency–Inverse Document Frequency (TF-IDF) vectorization, which extracts unigram and bigram tokens while applying sublinear frequency scaling and  $L_2$  normalization.

### 3.3.3 ANALYSIS OF CLASS VARIABLE

The target variable `medical_specialty` contains 40 distinct medical specialty categories. The class distribution exhibits severe imbalance, ranging from 1,103 reports in Surgery to only 2 reports in Autopsy and Executive Evaluation. To ensure fair multi-class evaluation, stratified train-test splitting (80:20) is utilized to preserve class proportions, and ensemble voting consensus is deployed to avoid model bias toward majority classes.

### 3.4 ANALYSIS OF ALGORITHMS

This project employs supervised machine learning algorithms to classify clinical transcription narratives into 40 distinct medical specialties. Multiple classification algorithms are considered to evaluate their effectiveness in distinguishing domain-specific clinical vocabulary based on extracted TF-IDF feature vectors. The proposed algorithms include Support Vector Machine (Linear SVM), Random Forest (RF), Logistic Regression (LR), and Multinomial Naive Bayes (MNB), integrated through a Voting Ensemble Classifier. These algorithms were chosen because they offer strong predictive performance in high-dimensional sparse text domains, provide architectural diversity, and execute efficiently on standard computing hardware without requiring specialized GPU accelerators.

#### 3.4.1 DETAILED STUDY REPORT OF THE PROPOSED ALGORITHMS

The proposed system evaluates diverse machine learning estimators alongside statistical feature extraction to construct a robust specialty classification engine. Each algorithm offers distinct theoretical properties, mathematical formulations, and operational characteristics when applied to high-dimensional clinical text spaces.

##### 1. Support Vector Machine (SVM)

Support Vector Machine (SVM) is a robust supervised learning algorithm founded on statistical learning theory and structural risk minimization principles. It constructs an optimal decision boundary (hyperplane) in a high-dimensional feature space to separate target classes while maximizing the geometric margin between them. Linear SVM (Linear SVC) is exceptionally well suited for text categorization where sparse TF-IDF vectors offer natural linear separability.

**A. Theoretical Foundation and Optimization Model** Given a training dataset of clinical transcription records:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\} \quad (1)$$

where  $x_i \in \mathbb{R}^d$  represents the sparse TF-IDF feature vector across  $d$  vocabulary terms and  $y_i \in \{1, 2, \dots, K\}$  denotes the target medical specialty ( $K = 40$ ), the linear decision hyperplane is defined as:

$$w^T x + b = 0 \quad (2)$$

where  $w \in \mathbb{R}^d$  is the weight vector normal to the hyperplane and  $b \in \mathbb{R}$  is the bias scalar. To handle real-world clinical vocabulary noise and lexical overlap between specialties, a soft-margin optimization formulation is applied using non-negative slack variables  $\xi_i \geq 0$ :

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \quad (3)$$

subject to the linear constraints:

$$y_{i,k}(w_k^T x_i + b_k) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i = 1, \dots, N \quad (4)$$

where  $C > 0$  is the regularization parameter controlling the trade-off between maximizing the geometric margin width ( $\frac{2}{\|w\|}$ ) and minimizing training classification error.

**B. Dual Formulation and Multi-Class Decision Function** By introducing Lagrange multipliers  $\alpha_i \geq 0$ , the primal optimization problem is transformed into its Wolfe dual representation:

$$\max_{\alpha} \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad (5)$$

subject to  $0 \leq \alpha_i \leq C$  and  $\sum_{i=1}^N \alpha_i y_i = 0$ . For high-dimensional sparse text vectors, the Linear Kernel  $K(x_i, x_j) = x_i^T x_j$  is utilized.

To support 40 medical specialties, a One-vs-Rest (OvR) strategy is implemented, training 40 independent binary classifiers. The decision function evaluating an incoming clinical transcription  $x$  is:

$$f_k(x) = w_k^T x + b_k \quad (6)$$

The predicted medical specialty  $\hat{y}$  corresponds to the class with the maximum decision score:

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} (w_k^T x + b_k) \quad (7)$$

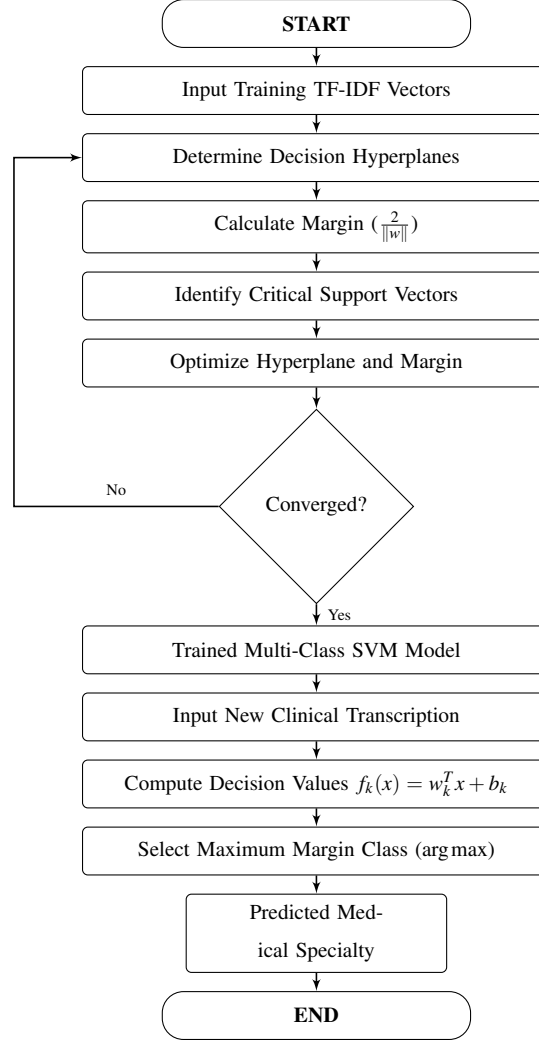
**C. Hyperparameter Tuning and Clinical Relevance** The predictive performance of Linear SVM depends primarily on optimizing the regularization parameter  $C$ . Linear SVM excels in clinical transcription classification due to its immunity to high-dimensional overfitting, low memory footprint during inference, and reliance solely on critical support vectors along deci-

sion margins.

**D. Probability Calibration for Soft-Voting Integration** Standard Linear Support Vector Classifiers (LinearSVC) compute uncalibrated signed geometric distances to separating hyperplanes, which do not directly provide posterior probability distributions (`predict_proba`). Because the proposed ensemble relies on soft-voting probability aggregation, the Linear SVM decision scores are calibrated to produce class probability estimates before soft-voting aggregation. This is achieved using Scikit-learn’s `CalibratedClassifierCV`, which applies Platt scaling via an internal cross-validated logistic sigmoid transformation:

$$P(y = k | x) = \frac{1}{1 + \exp(A \cdot f_k(x) + B)} \quad (8)$$

where scalar parameters  $A$  and  $B$  are estimated through maximum likelihood on internal validation folds. Probability calibration maps decision boundary margins monotonically into valid multi-class probability distributions, enabling Linear SVM to participate effectively in soft-voting probability aggregation alongside inherently probabilistic estimators.



**Figure 7:** Activity Diagram of Support Vector Machine

## 2. Random Forest (RF)

Random Forest is a high-performance ensemble learning algorithm based on decision tree aggregation. It constructs a multitude of uncorrelated decision trees during training and outputs the ensemble consensus. By combining Bootstrap Aggregation (Bagging) with random feature subspace selection, Random Forest mitigates variance without increasing prediction bias.

**A. Architectural Mechanics and Ensemble Formulation** Given a training dataset of  $N$  clinical transcription records, Random Forest generates  $B$  bootstrap samples by sampling with replacement. For each bootstrap sample, an individual decision tree  $T_b(x)$  is trained. At each split node of every tree, a random subset of  $m = \sqrt{d}$  TF-IDF features is evaluated to identify the optimal split.

The final specialty class prediction for an input vector  $x$  is derived through majority voting:

$$\hat{y} = \text{mode}(T_1(x), T_2(x), \dots, T_B(x)) \quad (9)$$

Alternatively, posterior class probabilities are computed as the average over all  $B$  decision trees:

$$P(y = k | x) = \frac{1}{B} \sum_{b=1}^B P_b(y = k | x) \quad (10)$$

**B. Node Splitting Criteria and Impurity Measures** Decision tree node splits are evaluated using impurity metrics such as Gini Impurity or Information Gain (Entropy).

- **Gini Impurity ( $G$ ):**

$$G = 1 - \sum_{k=1}^K p_k^2 \quad (11)$$

where  $p_k$  represents the proportion of instances belonging to specialty class  $k \in \{1, \dots, 40\}$  at a given node.

- **Entropy ( $H$ ) and Information Gain ( $IG$ ):**

$$H(S) = - \sum_{k=1}^K p_k \log_2(p_k) \quad (12)$$

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) \quad (13)$$

**C. Feature Importance Analysis and Clinical Interpretability** A major benefit of Random Forest is its capacity to quantify feature importance using Mean Decrease in Impurity (MDI). The importance index  $I(A_j)$  of TF-IDF feature  $A_j$  across all  $B$  decision trees is computed as:

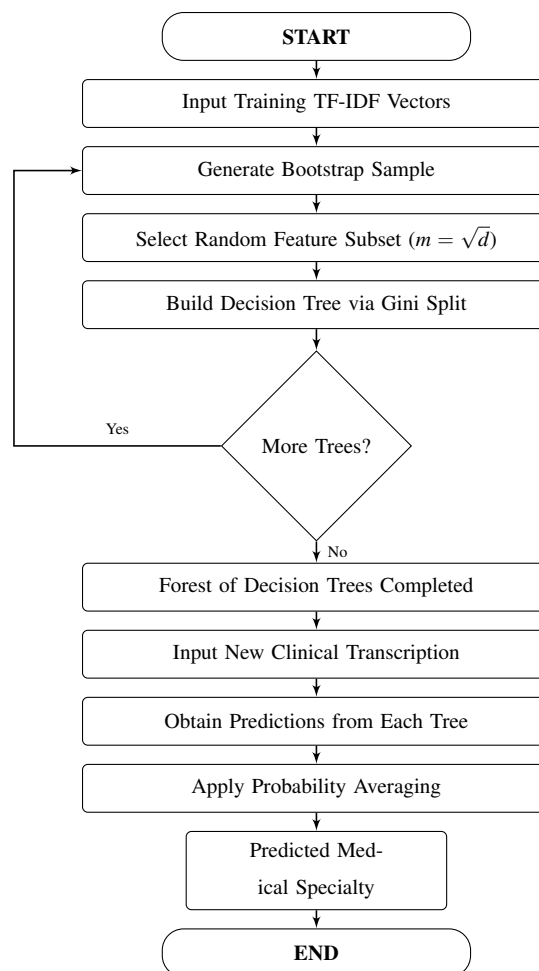
$$I(A_j) = \frac{1}{B} \sum_{b=1}^B \sum_{n \in N_b(A_j)} \Delta G(n) \quad (14)$$

where  $\Delta G(n)$  is the Gini impurity reduction achieved at node  $n$  by splitting on feature  $A_j$ . In clinical NLP, this enables identification and ranking of the most discriminative medical keywords for each specialty.

**D. Hyperparameter Optimization** The primary hyperparameters tuned for Random Forest include:

- `n_estimators` ( $B$ ): Number of decision trees (typically 100 to 300).
- `max_depth`: Maximum depth of decision trees to prevent single-tree memorization.
- `min_samples_split`: Minimum samples required to split an internal node.
- `min_samples_leaf`: Minimum samples required at a terminal leaf node.

Random Forest captures complex non-linear keyword interactions that linear models may overlook, adding structural diversity to the ensemble.



**Figure 8:** Activity Diagram of Random Forest

### 3. Logistic Regression (LR)

Logistic Regression is a foundational statistical learning algorithm for classification problems. Belonging to the family of Generalized Linear Models (GLMs), it is extended to multi-class

classification using the multinomial Softmax formulation, modeling posterior probabilities directly.

**A. Probabilistic Model and Multinomial Softmax Transformation** For an input TF-IDF vector  $x \in \mathbb{R}^d$  and  $K = 40$  medical specialty classes, the algorithm computes linear logit scores  $z_k = w_k^T x + b_k$ . The posterior probability that transcription  $x$  belongs to specialty  $k$  is given by the Softmax function:

$$P(y = k | x) = \frac{\exp(w_k^T x + b_k)}{\sum_{j=1}^K \exp(w_j^T x + b_j)} \quad (15)$$

where  $w_k \in \mathbb{R}^d$  is the weight vector associated with specialty  $k$  and  $b_k \in \mathbb{R}$  is the class-specific bias term.

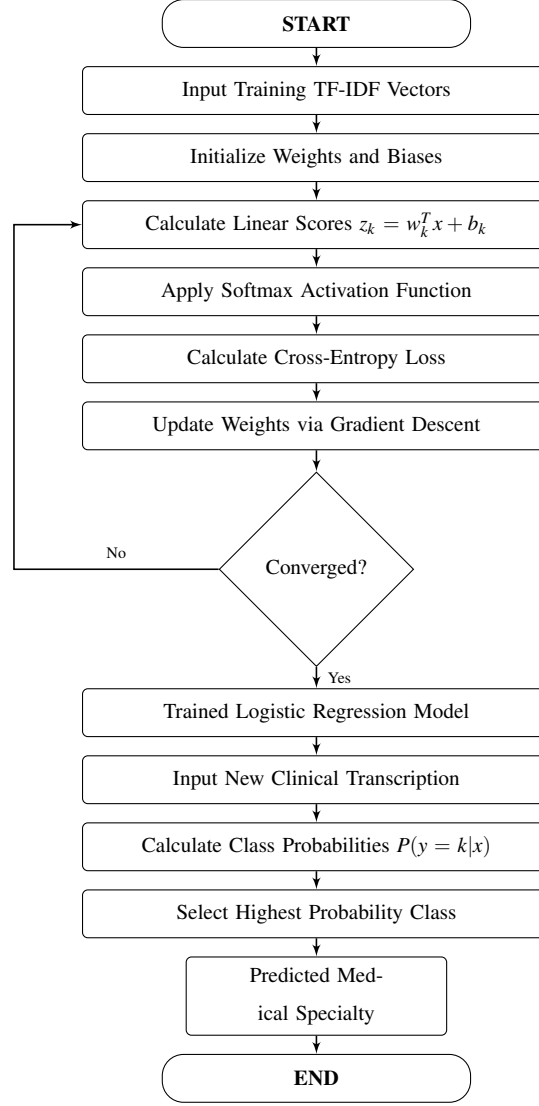
**B. Maximum Likelihood Estimation and Loss Function** Model parameter estimation is performed by minimizing the Categorical Cross-Entropy (Negative Log-Likelihood) loss function over  $N$  training samples, augmented with an  $L_2$  Ridge penalty:

$$J(W, b) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \mathbb{I}(y_i = k) \ln P(y_i = k | x_i) + \frac{\lambda}{2} \sum_{k=1}^K \|w_k\|_2^2 \quad (16)$$

where  $\mathbb{I}(\cdot)$  is the indicator function, and  $\lambda > 0$  controls the regularisation strength to prevent coefficient explosion on sparse text inputs. Parameters are optimized via gradient descent algorithms:

$$w_k^{(t+1)} = w_k^{(t)} - \alpha \nabla_{w_k} J(W, b) \quad (17)$$

**C. Clinical Relevance and Interpretability** Logistic Regression serves as an essential baseline model due to its high interpretability. The exponentiated weight coefficients  $\exp(w_{k,j})$  represent odds ratios for clinical terms, providing quantitative insight into which medical keywords escalate the probability of assigning a transcription to a specific specialty.



**Figure 9:** Activity Diagram of Logistic Regression

#### 4. Multinomial Naive Bayes (MNB)

Multinomial Naive Bayes is a specialized probabilistic classification algorithm rooted in Bayes' Theorem, specifically formulated for discrete word counts and term frequency distributions in document categorization tasks.

**A. Probabilistic Foundation and Bayes' Theorem** The algorithm computes the posterior probability of clinical transcription  $x = (x_1, \dots, x_m)$  belonging to medical specialty class  $c$  using Bayes' rule alongside the assumption of conditional feature independence given the class label:

$$P(c | x) \propto P(c) \prod_{i=1}^m P(x_i | c) \quad (18)$$

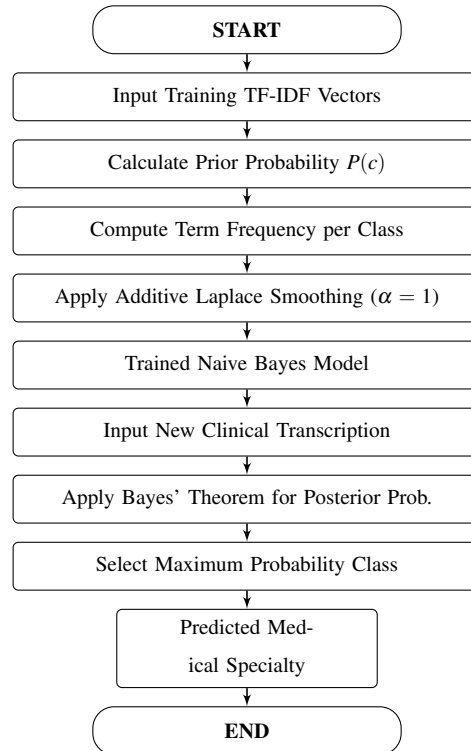
where  $P(c)$  is the prior probability of specialty  $c$  (derived from training class proportions) and  $P(x_i | c)$  is the class-conditional feature likelihood.

**B. Feature Likelihood and Laplace Smoothing** To prevent zero-probability penalties for clinical terms absent in the training split of a specific specialty, Laplace (additive) smoothing is applied:

$$P(x_i | c) = \frac{N_{ci} + \alpha}{N_c + \alpha|V|} \quad (19)$$

where  $N_{ci}$  is the cumulative occurrence count of term  $x_i$  in specialty class  $c$ ,  $N_c$  is the total token count across all records in class  $c$ ,  $|V|$  is the vocabulary size, and  $\alpha = 1.0$  is the smoothing hyperparameter.

**C. Computational Efficiency and Role in Medical Text Classification** Multinomial Naive Bayes executes in linear time with minimal memory overhead. It functions reliably on smaller clinical training subsets, providing an efficient baseline that contributes probabilistic diversity to the ensemble.



**Figure 10:** Activity Diagram of Multinomial Naive Bayes

## 5. TF-IDF Feature Vectorizer

Term Frequency–Inverse Document Frequency (TF-IDF) is a statistical vectorization technique that converts unstructured clinical text narratives into high-dimensional numerical feature vectors. In our proposed medical specialty classification system, TF-IDF operates as the core feature engineering component.

**A. Statistical Foundation and Mathematical Formulation** The vectorization process evaluates the relative importance of each term  $t$  within a specific document  $d$  against the entire clinical corpus  $D$ . Term Frequency, denoted as  $\text{TF}(t, d)$ , measures word occurrences scaled sublinearly:

$$\text{TF}(t, d) = 1 + \log(f_{t,d}) \quad (\text{if } f_{t,d} > 0) \quad (20)$$

Inverse Document Frequency, denoted as  $\text{IDF}(t, D)$ , computes the logarithmic inverse ratio:

$$\text{IDF}(t, D) = \log \left( \frac{1 + |D|}{1 + |\{d \in D : t \in d\}|} \right) + 1 \quad (21)$$

The composite TF-IDF weight is:

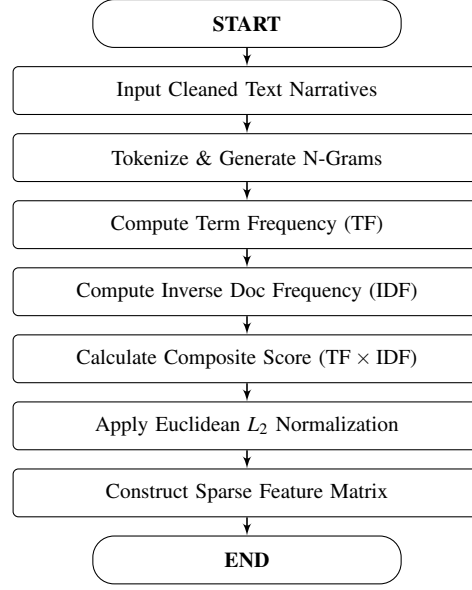
$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (22)$$

To ensure comparability across documents of varying lengths, Euclidean  $L_2$  normalization is applied:

$$v_{\text{norm}} = \frac{v}{\|v\|_2} = \frac{v}{\sqrt{\sum_i v_i^2}} \quad (23)$$

**B. N-Gram Extraction and Sparsity Optimization** The vectorizer extracts unigrams and bigrams, capturing both isolated medical terms (e.g., "appendectomy") and multi-word phrases (e.g., "coronary artery"). A minimum document frequency threshold filters out rare typos, yielding a normalized, highly efficient sparse matrix.

**C. Clinical Term Weighting and Down-Weighting Common Words** Through this formulation, terms appearing across almost all reports receive low weights, whereas specialty-specific diagnostic vocabulary receives high discriminative weights.



**Figure 11:** Activity Diagram of TF-IDF Feature Vectorizer

## 6. Voting Ensemble Classifier

The Voting Ensemble Classifier is the final meta-classification mechanism in the proposed system. Rather than relying on a single learning algorithm that may exhibit inductive biases across imbalanced specialties, the ensemble combines predictions from diverse candidate models (Logistic Regression, Linear SVM, Random Forest, and Multinomial Naive Bayes) to generate a consensus decision.

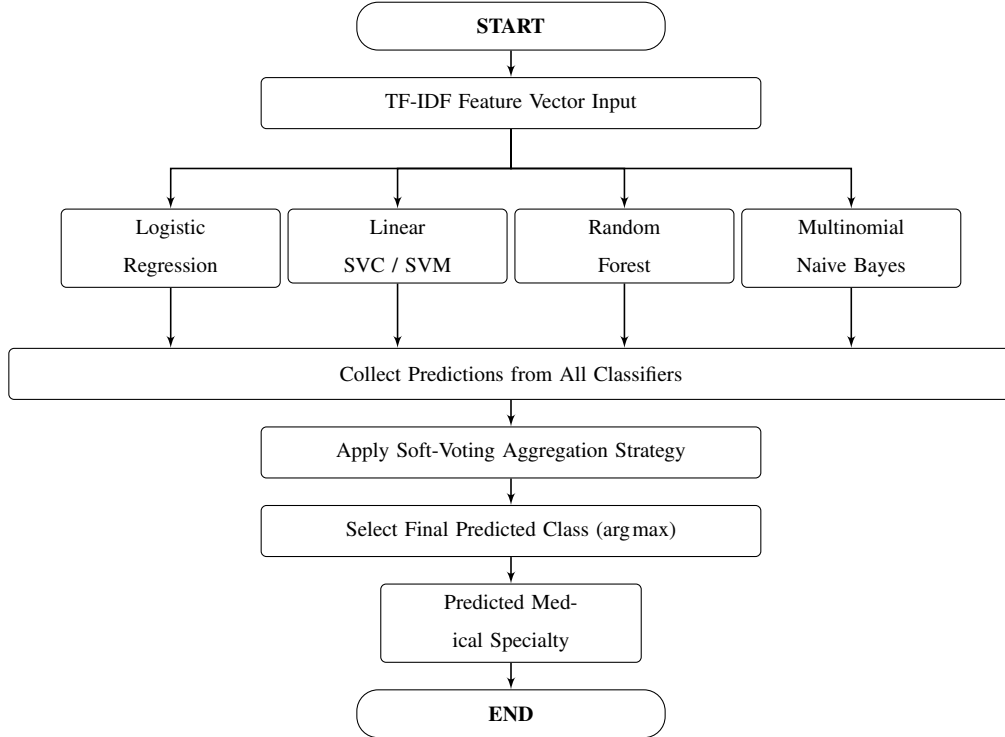
**A. Ensemble Mechanics and Soft-Voting Aggregation** Soft voting requires calibrated class probability estimates ( $P(y = k | x)$ ) across all candidate base estimators. While Logistic Regression, Random Forest, and Multinomial Naive Bayes intrinsically output class probabilities, standard Linear SVM outputs uncalibrated margin distances. Therefore, the Linear SVM decision scores are calibrated using `CalibratedClassifierCV` to produce reliable posterior probability estimates before soft-voting aggregation. In the soft-voting configuration, each calibrated base classifier  $m \in \{1, \dots, M\}$  generates a posterior class probability distribution  $P_m(y = k | x)$  indicating the likelihood that a given clinical transcription  $x$  belongs to medical specialty  $k$ . The ensemble then computes the arithmetic mean of these posterior probabilities across all  $M$  estimators:

$$P_{\text{ensemble}}(y = k | x) = \frac{1}{M} \sum_{m=1}^M P_m(y = k | x) \quad (24)$$

**B. Final Multi-Class Specialty Prediction** The final predicted medical specialty  $\hat{y}$  is selected as the category that maximizes the aggregated ensemble probability:

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} P_{\text{ensemble}}(y = k | x) \quad (25)$$

**C. Variance Reduction and Synergy of Diverse Estimators** By integrating linear probabilistic modeling (Logistic Regression), maximum-margin geometric boundaries (Linear SVM), non-linear decision trees (Random Forest), and conditional frequency modeling (Multinomial Naive Bayes), the ensemble offsets the individual weaknesses of each classifier, reduces predictive variance, and improves overall classification stability across all 40 medical specialties.



**Figure 12:** Activity Diagram of Voting Ensemble Classifier

### Working of the Proposed Algorithms

The proposed machine learning classification framework will execute through the following nine sequential steps:

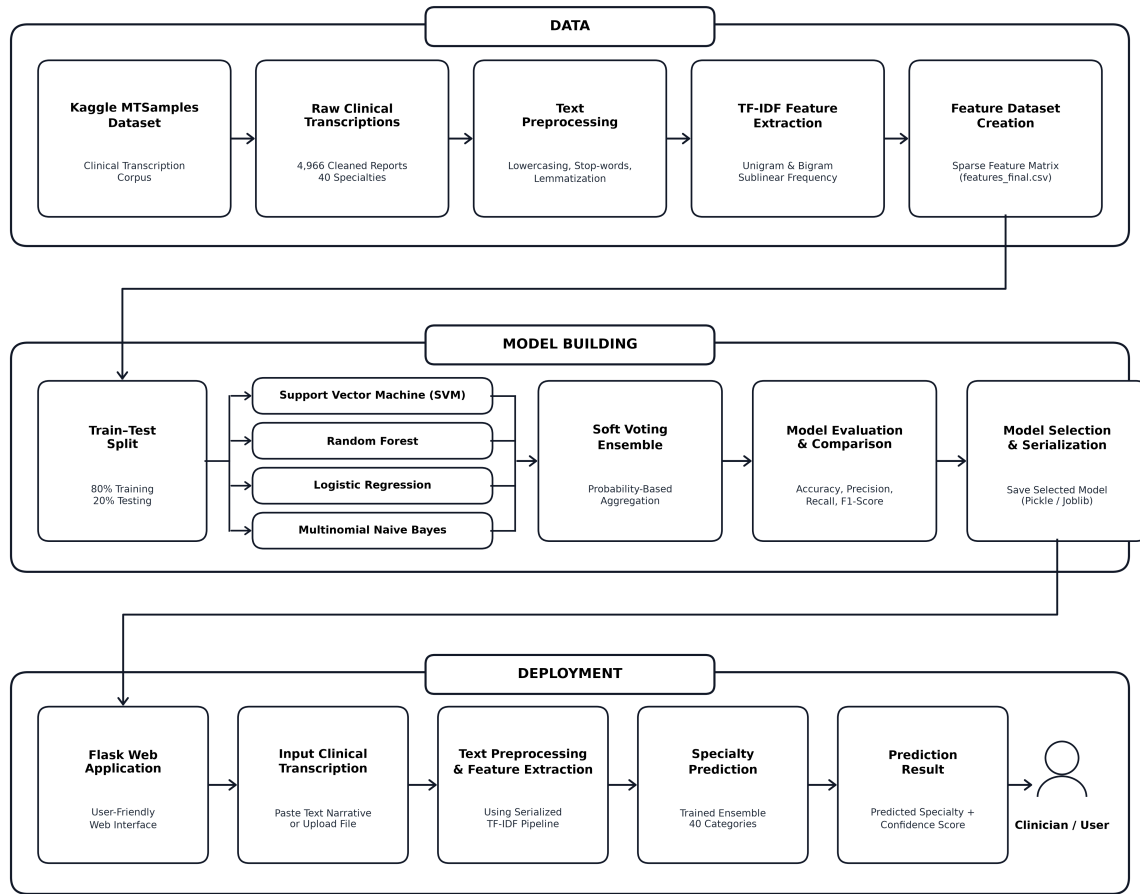
1. **Dataset Ingestion:** Ingesting the raw Kaggle MTSamples dataset containing 4,999 clinical records across 40 medical specialties.
2. **Data Cleaning:** Auditing and removing records with missing clinical transcriptions (33

null records), handling missing keywords (1,068 records), and standardizing text to lower case.

3. **Text Preprocessing:** Eliminating punctuation marks, digits, symbols, and domain stop-words, followed by WordNet morphological lemmatization.
4. **TF-IDF Feature Extraction:** Transforming preprocessed narratives into a high-dimensional,  $L_2$ -normalized sparse feature matrix via unigrams and bigrams with sublinear scaling.
5. **Stratified Train-Test Splitting:** Partitioning the feature matrix and target labels into an 80% training set and a 20% testing set using stratified sampling to preserve class proportion distributions across all 40 medical specialties.
6. **Independent Training of Candidate Classifiers:** Fitting candidate supervised learning algorithms (Linear Support Vector Machine with probability calibration, Random Forest, Logistic Regression, and Multinomial Naive Bayes) independently on the training partition.
7. **Hyperparameter Optimization:** Tuning regularisation parameters, smoothing factors, and estimators via grid search cross-validation.
8. **Voting Ensemble Aggregation:** Combining calibrated posterior class probability distributions generated by the candidate classifiers through soft-voting consensus aggregation.
9. **Medical Specialty Prediction:** Evaluating new clinical transcriptions, calculating aggregated ensemble probabilities, and outputting the top predicted medical specialty along with classification confidence scores.

### 3.5 PROJECT PIPELINE

The proposed Medical Specialty Classification system follows a structured pipeline that transforms raw clinical transcription narratives into reliable medical specialty predictions using machine learning. The pipeline begins with clinical text data acquisition, followed by text preprocessing, TF-IDF feature extraction, machine learning model development and voting ensemble aggregation, and finally medical specialty prediction through a user-friendly Flask web application. Each stage plays an important role in ensuring accurate and reliable classification across all 40 medical specialties.



**Figure 13:** Project Pipeline Diagram of Medical Specialty Classification System

The project pipeline consists of the following stages:

1. **Dataset:** The Kaggle MTSamples Medical Transcription Dataset containing 4,966 cleaned clinical reports across 40 medical specialties is used as the input corpus.
2. **Exploratory Data Analysis (EDA):** The distribution of transcription lengths, medical specialty frequencies, vocabulary counts, and class imbalances is systematically analyzed and statistically profiled.
3. **Text Preprocessing:** Unstructured clinical transcription narratives undergo thorough text cleaning, punctuation removal, lowercasing, medical stop-word filtering, and WordNet morphological lemmatization.
4. **Feature Extraction:** Term Frequency–Inverse Document Frequency (TF-IDF) feature extraction is applied with sublinear term-frequency scaling, unigram and bigram vocabulary extraction, and Euclidean  $L_2$  vector normalization.

5. **Feature Dataset Creation:** The extracted numerical representations are compiled into a structured, high-dimensional sparse feature matrix paired with mapped categorical target specialty labels.
6. **Train–Test Split:** The prepared feature dataset is divided using stratified sampling into an 80% training partition and a 20% testing partition to strictly preserve minority and majority class proportions across all 40 medical specialties.
7. **Machine Learning Model Development:** Multiple supervised machine learning algorithms, including Support Vector Machine (SVM with probability calibration), Random Forest (RF), Logistic Regression (LR), and Multinomial Naive Bayes (MNB), will be trained in parallel using the extracted features.
8. **Hyperparameter Tuning:** Grid search cross-validation will be performed to optimize critical hyperparameters, including regularisation strengths, smoothing parameters, and tree estimators across candidate models.
9. **Soft Voting Ensemble Formulation:** A Soft Voting Ensemble will be formulated using probability-based aggregation to synthesize calibrated class probability distributions from all candidate models, mitigating individual estimator variance.
10. **Model Evaluation and Comparison:** All trained candidate models and the Soft Voting Ensemble will be rigorously evaluated using standard multi-class classification metrics: Accuracy, Precision, Recall (Sensitivity), and F1-Scores.
11. **Model Selection and Serialization:** The selected best-performing model—specifically the Soft Voting Ensemble paired with the fitted TF-IDF vectorizer—will be serialized using Joblib/Pickle for persistent deployment.
12. **Deployment and Prediction:** The serialized model pipeline will be integrated into a lightweight Flask web application, enabling healthcare professionals to enter or upload patient clinical notes and receive real-time specialty predictions accompanied by classification confidence scores.

### 3.6 Project Timeline

The project timeline outlines the major activities planned for the development of the Medical Specialty Classification system, from initial dataset analysis and text preprocessing to machine

learning model development, voting ensemble integration, web application implementation, system testing, and final submission. The activities are organized according to the common project schedule and presentation milestones.

| Period / Date           | Planned Activity                                                                                              |
|-------------------------|---------------------------------------------------------------------------------------------------------------|
| Week 1–2                | Dataset Collection, Exploratory Data Analysis, and Initial Data Cleaning                                      |
| 21.07.2026              | Project Proposal Approval                                                                                     |
| Week 3–4                | Clinical Text Preprocessing, Medical Stop-word Removal, and Lemmatization                                     |
| Week 5                  | TF-IDF Feature Extraction and Feature Representation Analysis                                                 |
| 08.09.2026              | First Project Presentation                                                                                    |
| 09.09.2026              | Sprint Release I                                                                                              |
| Week 6                  | Candidate ML Model Training (Linear SVM, Random Forest, Logistic Regression, MNB) and Probability Calibration |
| 18.09.2026              | Sprint Release II                                                                                             |
| Week 7–8                | Soft Voting Ensemble Formulation, Grid Search Hyperparameter Tuning, and Initial Web Interface Design         |
| 29.09.2026 – 30.09.2026 | Interim Project Presentation                                                                                  |
| Week 9                  | Flask Web Application Integration, Route Setup, and Serialized Pipeline Deployment                            |
| 09.10.2026              | Sprint Release III                                                                                            |
| Week 10–11              | Multi-Class Evaluation, Confusion Matrix Analysis, Threshold Tuning, and End-to-End System Testing            |
| 22.10.2026 – 23.10.2026 | Final Project Presentation                                                                                    |
| 30.10.2026              | Final Report Submission                                                                                       |

**Table 9:** Project Timeline

### **3.7 FEASIBILITY ANALYSIS**

Feasibility analysis is conducted to evaluate the practicality of developing and implementing the proposed Medical Specialty Classification system. It assesses whether the project can be successfully developed using the available computational resources, technologies, budget, and time. The feasibility study ensures that the proposed solution is technically viable, economically affordable, operationally practical, and achievable within the designated project schedule.

#### **3.7.1 Technical Feasibility**

The proposed system is technically feasible because it utilizes widely adopted open-source software, established Natural Language Processing (NLP) libraries, and standard machine learning toolkits. The development environment consists of Python programming language, Google Colab, and Visual Studio Code, while libraries such as NLTK, Scikit-learn, Pandas, NumPy, Matplotlib, and Seaborn provide comprehensive functionality for text preprocessing, feature extraction, model training, and performance evaluation. The final system will be deployed using the Flask WSGI web framework, enabling healthcare practitioners to enter or upload clinical notes and obtain specialty predictions through an intuitive web interface. The required software tools are freely available and can be executed on standard CPU-based computing systems without requiring specialized high-performance GPU hardware.

#### **3.7.2 Economic Feasibility**

The proposed system is economically feasible because it relies entirely on open-source software and publicly available resources. The Kaggle MTSamples clinical transcription dataset used in this project is freely accessible, and all development tools, including Python, Google Colab, Flask, and the required machine learning libraries, are available without commercial licensing costs. Furthermore, because the classical machine learning workflow (TF-IDF paired with Linear SVM, Logistic Regression, Random Forest, and Naive Bayes) is computationally efficient, it eliminates the need for expensive cloud GPU instances or specialized hardware accelerators. Consequently, the overall development expenditure is minimal, making the project exceptionally cost-effective and suitable for academic research and clinical prototype deployment.

### **3.7.3 Operational Feasibility**

The proposed system is operationally feasible because it is designed to provide a simple and user-friendly interface for medical specialty prediction. Healthcare professionals and administrative personnel will be able to input clinical transcription narratives directly through the web application, after which the system will automatically preprocess the text, extract TF-IDF feature vectors, and generate specialty predictions using the trained Voting Ensemble model. The straightforward workflow minimizes user effort while providing an efficient, reliable, and accessible method for preliminary clinical categorization and automated medical workflow triaging.

### **3.7.4 Schedule Feasibility**

The project is considered schedule feasible because its development activities are planned in a structured and phased manner. The initial phases, including dataset acquisition, exploratory data analysis, data cleaning, and text preprocessing, have been planned and initiated according to the project schedule. The subsequent phases, such as machine learning model development, probability calibration, voting ensemble aggregation, performance evaluation, Flask web application deployment, and system testing, are scheduled sequentially according to the project timeline. The availability of suitable development tools, pre-existing datasets, and established algorithms supports the timely completion of the project within the academic schedule.

## **3.8 SYSTEM ENVIRONMENT**

The system environment specifies the software tools, libraries, and hardware resources required for the development and implementation of the proposed Medical Specialty Classification system. The project uses standard software tools and computing resources for text processing, feature extraction, machine learning model development, performance evaluation, and web application deployment.

### **3.8.1 Software Environment**

The software environment consists of Python and commonly used open-source libraries for Natural Language Processing, data analysis, machine learning, visualization, and web application development. Google Colab and Visual Studio Code are used for development, experimentation, and modular scripting, while Flask is used for deploying the trained model pipeline as an interactive web application. These tools are freely available and do not require specialized commercial software licenses. Table 10 details each software tool, its type, and its specific purpose in the proposed system.

| Software / Tool                 | Type                 | Purpose                                                                                                            |
|---------------------------------|----------------------|--------------------------------------------------------------------------------------------------------------------|
| Python                          | Programming Language | Primary programming language used for data preprocessing, feature extraction, model training, and backend scripts. |
| Google Colab                    | Cloud IDE            | Cloud-based interactive environment used for exploratory data analysis, model experimentation, and visualization.  |
| Visual Studio Code              | Development IDE      | Primary IDE for modular Python script development, project organization, and debugging.                            |
| L <sup>A</sup> T <sub>E</sub> X | Typesetting System   | Document preparation system used for compiling academic reports and formatted tables.                              |
| Pandas                          | Data Manipulation    | Handling structured CSV data, dataset filtering, null audits, and category mapping.                                |
| NumPy                           | Numerical Computing  | Efficient multi-dimensional array manipulation and mathematical operations.                                        |
| NLTK                            | NLP Toolkit          | Text tokenization, stop-word elimination, and WordNet morphological lemmatization.                                 |
| Scikit-learn                    | Machine Learning     | TF-IDF feature extraction, individual classifier training, hyperparameter tuning, and Voting Ensemble aggregation. |
| Matplotlib                      | Data Visualization   | Rendering statistical charts, class distribution plots, and performance curves.                                    |
| Seaborn                         | Statistical Graphics | High-level statistical visualization for categorical frequencies and confusion matrices.                           |
| WordCloud                       | Text Visualization   | Generating lexical frequency word clouds from clinical narratives.                                                 |
| Flask                           | Web Framework        | Lightweight WSGI framework for serving serialized model predictions via an interactive web interface.              |

**Table 10:** Software Environment and Tool Specifications

### 3.8.2 Hardware Environment

The proposed Medical Specialty Classification system does not require specialized hardware for its development or operation. The classical machine learning approach, combining TF-IDF feature extraction with ensemble supervised classifiers, is computationally lightweight compared to deep learning transformers. A standard laptop or desktop computer with sufficient processing capability, memory, and storage can be used for text preprocessing, feature extraction, model training, and Flask web application hosting. Table 11 outlines the minimum and recommended hardware configurations required for developing and running the proposed system.

| Hardware Component  | Minimum Requirement                     | Recommended Specification                                                                      |
|---------------------|-----------------------------------------|------------------------------------------------------------------------------------------------|
| Processor (CPU)     | Dual-Core Processor (2.0 GHz or higher) | Quad-Core / Octa-Core Intel Core i5/i7 or AMD Ryzen 5/7 (Multi-core CPU; no GPU required)      |
| RAM                 | 8 GB DDR4                               | 16 GB DDR4/DDR5 (Supports parallel cross-validation and multiple candidate estimators)         |
| Storage             | 20 GB available disk space              | 256 GB / 512 GB Solid State Drive (SSD) for rapid dataset read/write and virtual environments  |
| Operating System    | Windows 10 (64-bit)                     | Windows 10/11 (64-bit), Ubuntu Linux 20.04+, or macOS                                          |
| Internet Connection | Standard Broadband (5 Mbps)             | High-Speed Broadband Connection (For package installation, dataset download, and cloud backup) |

**Table 11:** Hardware Specifications and Environment Requirements

## References

- [1] A. M. Omar, M. Elhoseny, and A. M. Hassanien, “Clinical text classification with word representation features and machine learning algorithms,” *International Journal of Online and Biomedical Engineering (iJOE)*, vol. 19, no. 4, pp. 40–54, 2023.
- [2] H. Zhang, D. Zhu, H. Tan, M. Shafiq, and Z. Gu, “Medical specialty classification based on semiadversarial data augmentation,” *Hindawi Computational Intelligence and Neuroscience*, vol. 2023, pp. 1–14, 2023.
- [3] A. E. Blanchard, S. Gao, H.-J. Yoon, and G. Tourassi, “A keyword-enhanced approach to handle class imbalance in clinical text classification,” *IEEE Journal of Biomedical and Health Informatics*, vol. 26, no. 6, pp. 2796–2803, Jun. 2022.
- [4] T. Boyle, “Medical transcriptions dataset,” <https://www.kaggle.com/datasets/tboyle10/medicaltranscriptions>, 2019, accessed: 2026-08-03.